

Structured Query Language - SQL

Parte 5

QXD0011 - Fundamentos de Bancos de Dados

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro

victorpinheiro@ufc.br



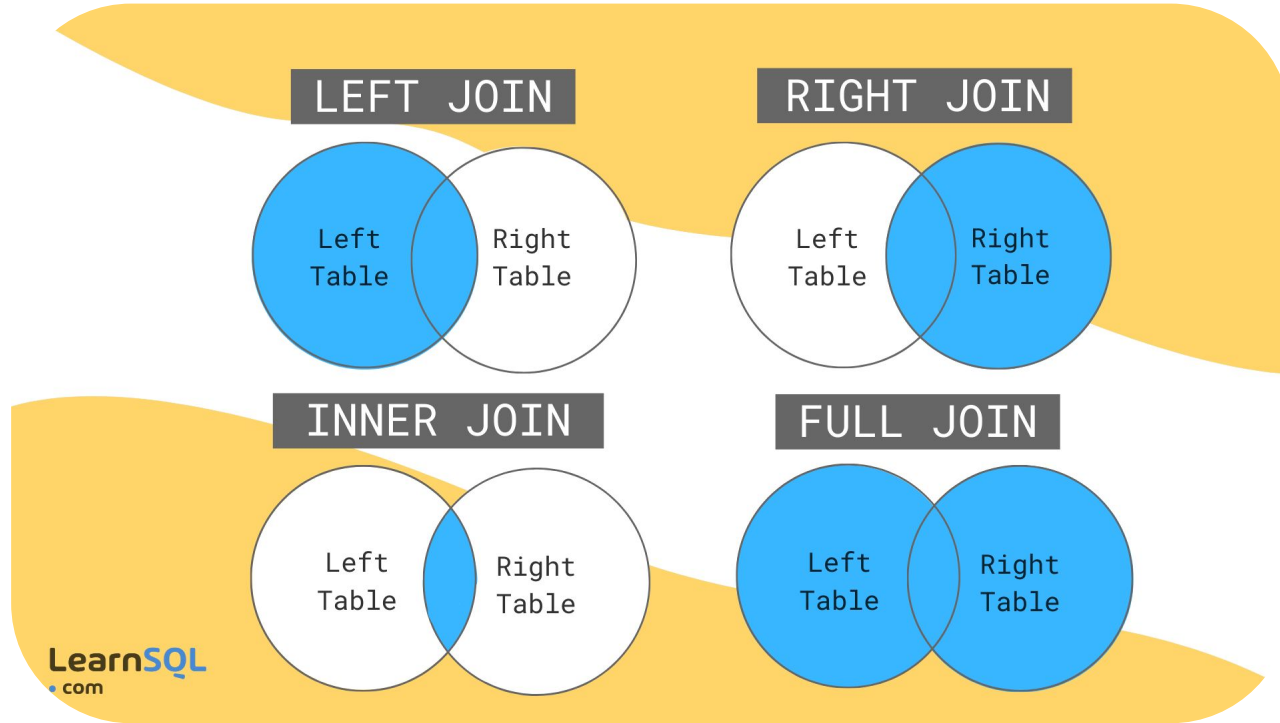
Agenda

- Junções
- Junções + Funções agregadas
- Tipos de Junções
 - Theta join
 - Equijoin
 - Natural Join
 - INNER JOIN
 - LEFT JOIN (LEFT OUTER JOIN)
 - RIGHT JOIN (RIGHT OUTER JOIN)
 - FULL JOIN (FULL OUTER JOIN)
 - CROSS JOIN
 - SELF JOIN
- Resumo dos Tipos de JOIN
- Formas de Junção em SQL Avançado

Junções

- Definição: Junções em SQL permitem combinar dados de duas ou mais tabelas com base em uma relação lógica entre elas (normalmente uma chave primária e uma chave estrangeira).
- Por quê usar? Para consultar informações distribuídas em diferentes tabelas, compondo resultados mais ricos.

Junções



Junções

- Usuários podem especificar uma tabela resultante de uma operação de junção na cláusula FROM de uma consulta.
 - Consulta a seguir tem uma única tabela de junção.

```
C1A: SELECT  Pnome, Unome, Endereco
          FROM    (FUNCIONARIO JOIN
                  DEPARTAMENTO
                  ON Dnr=Dnumero)
          WHERE   Dnome='Pesquisa';
```

Junções + Funções agregadas

- Funções agregadas são utilizadas para calcular valores a partir de conjuntos de registros (ex: COUNT, SUM, AVG, MAX, MIN).
- Exemplo clássico: Quantos funcionários há em cada departamento?

```
SELECT d.dnome, COUNT(f.cpf) AS qtd_funcionarios  
FROM departamento d  
LEFT JOIN funcionario f ON f.dnr = d.dnumero  
GROUP BY d.dnome;
```

Junções + Funções agregadas

- Achar a soma dos salários de todos os funcionários do departamento 'Pesquisa', bem como o salário máximo, o salário mínimo e a média dos salários nesse departamento.

```
C20: SELECT SUM (Salario), MAX (Salario),
           MIN (Salario), AVG (Salario)
FROM      (FUNCIONARIO JOIN
           DEPARTAMENTO ON Dnr=Dnumero)
WHERE     Dnome='Pesquisa';
```

Tipos de Junções

- Theta join permite comparações arbitrárias de relacionamentos como $=$, $\geq$, $\leq$, $>$, $<$.
- Equijoin é um theta join usando somente o operador de igualdade.
- Natural join é um equijoin em atributos que possuem o mesmo nome na relações usadas na junção.
 - Uma junção natural remove colunas duplicadas envolvidas na comparação.
 - Somente uma das colunas comparadas permanece no resultado da junção.

Theta Join (θ -join)

- É uma junção onde a condição pode ser qualquer operador relacional ($=$, $>$, $<$, $>=$, $<=$, $<>$), não só igualdade.
- Sintaxe: Usada mais em álgebra relacional e alguns bancos, mas no SQL é representada pelo JOIN ... ON com condição personalizada.
- Permite qualquer tipo de comparação, não só igualdade.

Exemplo

- Liste pares de funcionários e departamentos em que o salário do funcionário é maior que o número do departamento.

```
SELECT f.pnome, d.dnome  
FROM funcionario f  
JOIN departamento d ON f.salario > d.dnumero;
```

Equijoin

- É um caso especial do theta join, onde a comparação é apenas igualdade (=).
- Sintaxe: Usamos o ON tabela1.coluna = tabela2.coluna.
- Sempre usa igualdade para unir registros entre tabelas.

Exemplo

- Liste nome do funcionário e nome do departamento onde ele trabalha.

```
SELECT f.pnome, d.dnome  
FROM funcionario f  
JOIN departamento d ON f.dnr = d.dnumero;
```

Natural Join

- É um tipo especial de equijoin em que a junção é feita automaticamente nos atributos que possuem o mesmo nome em ambas as tabelas. O resultado elimina uma das colunas duplicadas.
- Sintaxe: `SELECT ... FROM tabela1 NATURAL JOIN tabela2`
- Evita colunas duplicadas.
- Automático: não precisa escrever a condição, mas pode ser perigoso se houver nomes iguais sem intenção.

Exemplo

- Supondo que funcionario.dnr e departamento.dnumero se chamassem dnr em ambas:
- O SQL faz a junção usando todos os campos com mesmo nome (no caso, dnr), e no resultado aparece só uma coluna dnr.

```
SELECT *
FROM funcionario
NATURAL JOIN departamento;
```

Natural Join

- Natural JOIN sobre duas relações R e S, nenhuma condição de junção é especificada. Cria-se uma condição EQUIJOIN implícita para cada par de atributos com o mesmo nome.

```
select pnome, nome_dep
from funcionario
    natural join departamento as dep
(nome_dep, dnr, cpf_gerente, data_inicio_gerente);
```

Natural Join

```
C1B: SELECT  Pnome, Unome, Endereco
FROM      (FUNCIONARIO NATURAL JOIN
           (DEPARTAMENTO AS DEP
            (Dnome, Dnr, Cpf_gerente, Data_inicio_gerente)))
WHERE     Dnome='Pesquisa';
```


Comparativo

Tipo	Operador	Exemplo de condição	Colunas duplicadas?	Uso prático
Theta Join	Qualquer	$A.id > B.id$	Sim	Comparações arbitrárias
Equijoin	Igualdade =	$A.id = B.id$	Sim	Junções típicas
Natural Join	Igualdade =	Usa todos os campos de mesmo nome	Não	Evitar redundância, automatizar condição

INNER JOIN

- O INNER JOIN é um tipo de junção em SQL que serve para combinar dados de duas tabelas diferentes. Ele faz isso com base em uma condição de correspondência, normalmente uma coluna que as duas tabelas têm em comum (geralmente uma chave primária e uma chave estrangeira).
- O resultado de um INNER JOIN são apenas as linhas que têm correspondência nas duas tabelas.
- O INNER JOIN retorna apenas os registros que possuem correspondência nas duas tabelas. Se não houver correspondência, o registro é ignorado no resultado.

Situação do mundo real

- Imagine que você tem uma lista de funcionários e uma lista de departamentos.
- Você quer saber apenas os funcionários que já estão alocados em algum departamento.

INNER JOIN

Tabela: funcionario

id	nome	dnr
1	Ana	10
2	Bruno	20
3	Carla	30
4	Daniel	40

Tabela: departamento

dnumero	dnome
10	Compras
20	Vendas
30	RH

INNER JOIN

```
SELECT *  
FROM funcionario f  
INNER JOIN departamento d ON f.dnr = d.dnumero;
```

O que acontece aqui?

- O SQL pega cada linha da tabela "funcionario" e procura uma linha correspondente na tabela "departamento" onde o valor de f.dnr (departamento do funcionário) é igual a d.dnumero (número do departamento).
- Só aparecem no resultado as linhas onde essa correspondência existe.

INNER JOIN

Resultado

nome	dnome
Ana	Compras
Bruno	Vendas
Carla	RH

Note que Daniel não aparece, pois o departamento 40 não existe na tabela departamento.

Resumindo INNER JOIN...

- INNER JOIN: só traz registros quando existe relação nas duas tabelas.
- Se não houver "par" na outra tabela, o dado não aparece.
- É o tipo de JOIN mais usado quando só queremos dados "casados".

LEFT JOIN (LEFT OUTER JOIN)

- O LEFT JOIN pega TODOS os registros da tabela da ESQUERDA (a primeira tabela na consulta).
- Se houver correspondência na tabela da DIREITA, ele mostra os dados juntos.
- Se não houver correspondência, os campos da tabela da direita aparecem como NULL.
- Nunca “some” dados da tabela da esquerda!

Situação do mundo real

- Você quer listar todos os funcionários, mesmo os que ainda não estão alocados em nenhum departamento.

LEFT JOIN (LEFT OUTER JOIN)

Tabela: funcionario

id	nome	dnr
1	Ana	10
2	Bruno	20
3	Carla	30
4	Daniel	40

Tabela: departamento

dnumero	dnome
10	Compras
20	Vendas
30	RH

LEFT JOIN (LEFT OUTER JOIN)

```
SELECT f.nome, d.dnome  
FROM funcionario f  
LEFT JOIN departamento d ON f.dnr = d.dnumero;
```

O que acontece aqui?

- O SQL vai passar linha por linha na tabela “funcionario”.
- Para cada funcionário, tenta achar um departamento com o mesmo número (f.dnr = d.dnumero).
- Se encontrar, traz o nome do departamento.
- Se NÃO encontrar, coloca NULL nos campos do departamento.

LEFT JOIN (LEFT OUTER JOIN)

Resultado

nome	dnome
Ana	Compras
Bruno	Vendas
Carla	RH
Daniel	NULL

Note que Daniel aparece, mas o nome do departamento é NULL, pois não há departamento 40

Resumindo LEFT JOIN (LEFT OUTER JOIN)...

- LEFT JOIN: Garante que TODOS os registros da tabela da esquerda vão aparecer.
- Se não houver correspondência na tabela da direita, os campos dessa tabela aparecem como NULL.
- Útil para encontrar dados “sem par”, como funcionários sem departamento, clientes sem pedidos, etc.

RIGHT JOIN (RIGHT OUTER JOIN)

- O RIGHT JOIN faz o oposto do LEFT JOIN:
 - Ele retorna TODOS os registros da tabela da DIREITA (a segunda tabela citada na query).
 - Traz também os registros correspondentes da tabela da ESQUERDA, quando existe relação.
 - Se não houver correspondência na tabela da esquerda, os campos dela aparecem como NULL.
- Ou seja: nunca “some” dados da tabela da direita!

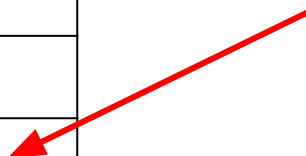
Situação do mundo real

- Você quer listar todos os departamentos, mesmo que ainda não tenham nenhum funcionário alocado.

Tabela: departamento

dnumero	dnome
10	Compras
20	Vendas
30	RH
50	Marketing

Vamos adicionar mais um departamento sem funcionários



RIGHT JOIN (RIGHT OUTER JOIN)

```
SELECT f.nome, d.dnome  
FROM funcionario f  
RIGHT JOIN departamento d ON f.dnr = d.dnumero;
```

O que acontece aqui?

- O SQL vai passar linha por linha na tabela “departamento” (tabela da direita).
- Para cada departamento, tenta achar um funcionário com $f.dnr = d.dnumero$.
- Se encontrar, traz o nome do funcionário junto.
- Se NÃO encontrar, coloca NULL nos campos do funcionário.

RIGHT JOIN (RIGHT OUTER JOIN)

Resultado

nome	dnome
Ana	Compras
Bruno	Vendas
Carla	RH
NULL	Marketing

O departamento Marketing aparece, mesmo sem funcionários, mas o nome do funcionário fica NULL.

Resumindo RIGHT JOIN (RIGHT OUTER JOIN) ...

- RIGHT JOIN: Garante que todos os registros da tabela da DIREITA (a que está após o JOIN) apareçam.
- Se não houver correspondência na tabela da esquerda, os campos dessa tabela ficam NULL.
- Muito útil para ver, por exemplo, departamentos sem funcionários, cursos sem alunos, produtos sem vendas, etc.

FULL JOIN (FULL OUTER JOIN)

- O FULL JOIN junta tudo:
 - Ele retorna TODOS os registros das duas tabelas.
 - Onde houver correspondência, ele traz os dados “casados”.
 - Onde não houver correspondência, ele preenche os campos da tabela que ficou sem “par” com NULL.
- O FULL JOIN retorna todos os registros das duas tabelas. Onde houver correspondência, junta os dados; onde não houver, completa com NULL.

FULL JOIN (FULL OUTER JOIN)

- Ele une o resultado do LEFT JOIN com o RIGHT JOIN:
 - Vai aparecer todo mundo das duas tabelas.
 - Se houver relação (funcionário com departamento), aparece junto.
 - Se alguém não tem par, aparece com os campos da outra tabela como NULL.

Situação do mundo real

- Você quer ver todos os funcionários e todos os departamentos, mesmo se algum funcionário não tiver departamento ou algum departamento não tiver funcionário.

FULL JOIN (FULL OUTER JOIN)

```
SELECT f.nome, d.dnome  
FROM funcionario f  
FULL JOIN departamento d ON f.dnr = d.dnumero;
```

O que acontece aqui?

- Todos os funcionários aparecem — mesmo Daniel, que não tem departamento.
- Todos os departamentos aparecem — mesmo Marketing, que não tem funcionário.
- Se não houver “par”, o campo correspondente fica NULL.

FULL JOIN (FULL OUTER JOIN)

Resultado

nome	dnome
Ana	Compras
Bruno	Vendas
Carla	RH
Daniel	NULL
NULL	Marketing

Todos aparecem, cada um com seus respectivos valores ou NULL onde não há correspondência.

CROSS JOIN

- O CROSS JOIN retorna todas as combinações possíveis entre os registros das duas tabelas. Isso é chamado de produto cartesiano.
- Situação do mundo real: Imagine que você quer combinar cada funcionário com todos os departamentos (por exemplo, para simular possíveis alocações).

CROSS JOIN

- Cada funcionário é combinado com cada departamento.

```
SELECT f.nome, d.dnome  
FROM funcionario f  
CROSS JOIN departamento d;
```

SELF JOIN

- O SELF JOIN é uma junção onde uma tabela é combinada consigo mesma. Usamos apelidos para diferenciar as "duas versões" da mesma tabela.
- Situação do mundo real: Suponha que cada funcionário tenha um gerente, que também está na tabela de funcionários. Você quer listar o nome do funcionário e o nome do seu gerente.

SELF JOIN

- Mostra o relacionamento de hierarquia dentro da própria tabela.

```
SELECT f1.pnome AS funcionario, f2.pnome AS  
supervisor  
FROM funcionario as f1  
JOIN  funcionario as f2 ON f1.cpf = f2.cpf_supervisor;
```

Resumo dos Tipos de JOIN

Tipo de JOIN	O que retorna?	Caso típico de uso
INNER JOIN	Apenas correspondências nas duas tabelas	Funcionários que já têm departamento
LEFT JOIN	Todos da esquerda, e os da direita se houver	Todos os funcionários, com/sem depto
RIGHT JOIN	Todos da direita, e os da esquerda se houver	Todos os departamentos, com/sem funcionário
FULL JOIN	Todos das duas tabelas, completos com NULL	Tudo, até os "órfãos"
CROSS JOIN	Todas as combinações possíveis	Simulações, combinações, análises
SELF JOIN	Tabela com ela mesma	Hierarquias, relações internas

Junções

- Múltiplas junções em uma mesma consulta.

```
select nome_dependente,pnome,dnome  
from dependente d left join funcionario f on d.fcpf=f.cpf left  
join departamento de on f.dnr = de.dnumero;
```

Junções

- A consulta lista todos os dependentes e, para cada um:
 - O nome do dependente,
 - O nome do funcionário responsável (se existir),
 - O nome do departamento desse funcionário (se existir).

- Se o dependente não tiver um funcionário responsável cadastrado (ou o funcionário não estiver mais na empresa), o nome do funcionário e o departamento vão aparecer como NULL.

- Se o funcionário existir, mas não estiver em nenhum departamento cadastrado, só o nome do departamento será NULL.

Exemplo de resultado

nome_dependente	pnome	dnome
Joãozinho	Maria	RH
Carol	João	Compras
Pedro	NULL	NULL
Lara	Lucas	NULL

- **dependente** (tabela base)
 - ← LEFT JOIN com **funcionario** (pode não existir → NULL)
 - ← LEFT JOIN com **departamento** (pode não existir → NULL)

Formas de Junção em SQL Avançado

- As junções (JOINS) são uma das ferramentas mais poderosas do SQL, permitindo combinar dados de duas ou mais tabelas com base em relações lógicas entre elas.
- Além das junções básicas (INNER, LEFT, RIGHT, FULL), existem formas avançadas de junção que ampliam as possibilidades de consulta, análise e integração de dados.
- Essas formas avançadas permitem resolver cenários complexos de relacionamento, análise temporal, buscas hierárquicas e até comparações entre conjuntos de dados.

Sintaxe da cláusula FROM

- FROM <tabela_fonte> [, ...n]
- <tabela_fonte> ::=
- nome_tabela [[AS] <qualificador>]
- | <tabela_fonte> <tipo_juncao> <tabela_fonte> ON <condicao_juncao>
- <tipo_juncao> ::=
- [INNER
- | LEFT [OUTER]
- | RIGHT [OUTER]
- | FULL [OUTER]
-] [<join_hint>]
- JOIN

Junção usando Subquery (Subconsulta)

- Exemplo: trazer o nome do funcionário e o nome do departamento onde trabalha, apenas para quem ganha acima da média do departamento.

```
SELECT f.pnome, d.dnome FROM funcionario f JOIN departamento d ON f.dnr = d.dnumero
WHERE f.salario > (
    SELECT AVG(f2.salario)
    FROM funcionario f2
    WHERE f2.dnr = f.dnr
);
```

Junção com Funções Agregadas em Subquery

- Exemplo: traga os departamentos e o total de salário pago, apenas para departamentos com mais de 5 funcionários.

```
SELECT d.dnome, SUM(f.salario) AS total_salario FROM departamento d
JOIN funcionario f ON f.dnr = d.dnumero GROUP BY d.dnome
HAVING COUNT(f.cpf) > 5;
```

SELF JOIN para hierarquias

- Exemplo: Quem são os subordinados de cada supervisor?

```
SELECT f1.pnome AS funcionario, f2.pnome AS supervisor  
FROM funcionario f1  
JOIN funcionario f2 ON f1.cpf_supervisor = f2.cpf;
```


Q1

- Liste todos os funcionários e, se houver, o nome do seu departamento (mesmo os que não têm departamento).

```
SELECT f.pnome AS funcionario, d.dnome AS departamento
FROM funcionario f
LEFT JOIN departamento d ON f.dnr = d.dnumero;
```

- **Por quê?**
 - LEFT JOIN garante que todos os funcionários aparecem, mesmo sem departamento.
 - Se um funcionário não tem departamento, o campo departamento será NULL.

Q2

- Mostre o nome dos departamentos e a quantidade de funcionários lotados em cada um (mesmo se não houver funcionários).

```
SELECT d.dnome AS departamento, COUNT(f.cpf) AS qtd_funcionarios
FROM departamento d
LEFT JOIN funcionario f ON f.dnr = d.dnumero
GROUP BY d.dnome;
```

- **Por quê?**
 - LEFT JOIN garante que todos os departamentos aparecem, mesmo que não tenham funcionários.
 - Departamentos sem funcionários terão qtd_funcionarios = 0.

Q3

- Traga o nome dos departamentos que não possuem funcionários.

```
SELECT d.dnome AS departamento  
FROM departamento d  
LEFT JOIN funcionario f ON f.dnr = d.dnumero  
WHERE f.cpf IS NULL;
```

- **Por quê?**
 - LEFT JOIN para pegar todos os departamentos.
 - A condição WHERE f.cpf IS NULL filtra apenas os departamentos sem correspondência, ou seja, sem funcionários.

Q4

- Liste os funcionários que têm o salário maior do que a média do seu departamento.

```
SELECT f.nome AS funcionario, f.salario, d.dnome AS departamento
FROM funcionario f JOIN departamento d ON f.dnr = d.dnumero
WHERE f.salario > (
    SELECT AVG(f2.salario)
    FROM funcionario f2
    WHERE f2.dnr = f.dnr
);
```

- **Por quê?**
 - Usamos um subselect para calcular a média salarial por departamento.
 - Seleciona apenas funcionários cujo salário é maior que a média do seu próprio departamento.

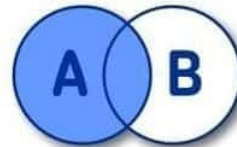
Q5

- Mostre todos os pares possíveis de funcionário e departamento (produto cartesiano).

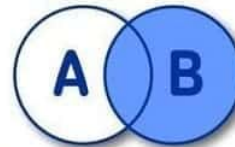
```
SELECT f.nome AS funcionario, d.dnome AS departamento
FROM funcionario f
CROSS JOIN departamento d;
```

- **Por quê?**
 - CROSS JOIN retorna todas as combinações possíveis de funcionário e departamento (produto cartesiano).

SQL JOINS



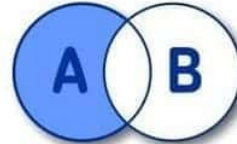
SELECT * FROM
A **LEFT** JOIN B
ON A.KEY = B.KEY



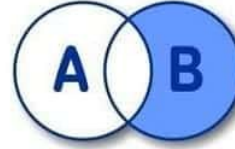
SELECT * FROM
A **RIGHT** JOIN B
ON A.KEY = B.KEY



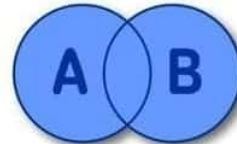
SELECT * FROM
A **INNER** JOIN B
ON A.KEY = B.KEY



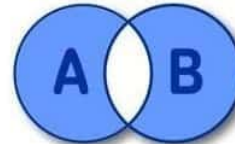
SELECT * FROM A
LEFT JOIN B
ON A.KEY = B.KEY
WHERE B.KEY IS NULL



SELECT * FROM A
RIGHT JOIN B
ON A.KEY = B.KEY
WHERE A.KEY IS NULL



SELECT * FROM A
FULL OUTER JOIN B
ON A.KEY = B.KEY



SELECT * FROM A
FULL OUTER JOIN B ON A.KEY =
B.KEY WHERE A.KEY IS
NULL OR B.KEY IS NULL

Bibliografia Básica

- ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 6 ed. Pearson/Addison-Wesley, 2011. ISBN: 9788579360855
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. Rio de Janeiro: Elsevier: Campus, 2012. ISBN 9788535245356
- HEUSER, C. A. Projeto de banco de dados. 6 ed. Bookman, 2009. ISBN: 9788577803828



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

